UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Metodología de Sistemas II

Trabajo Práctico Integrador

Evolución profesional de un sistema existente: calidad, deuda técnica, refactoring y revisión técnica

Sistema de Gestión de Solicitudes Académicas

Integrantes: Gevont Utmazian, Martín Maine
Comisión: M25 C4-08
Año académico: 2026

Fecha de entrega: 08/11/2026.

Informe técnico. El uso de IA se detalla en el apartado correspondiente.

Índice

1. Resumen ejecutivo 3

2. Introducción y objetivos 3

3. Sistema base y alcance 4

4. Calidad externa e interna 5

5. Deuda técnica y code smells 6

6. SOLID, Clean Code y decisiones 7

7. Refactoring seguro 8

8. Pruebas y verificación 12

9. Code review y criterios de aceptación 16

10. Conclusiones y mejoras futuras 18

11. Uso de inteligencia artificial 19

12. Referencias y recursos 20

Anexo A. Historial real de desarrollo 21

1. Resumen ejecutivo

Se construyó un prototipo local para gestionar solicitudes académicas.
La versión inicial resolvía los casos previstos, pero concentraba reglas y SQL.
El diagnóstico identificó ocho debilidades de legibilidad, cohesión y acoplamiento.
Se aplicaron cuatro refactorizaciones y se conservó la interfaz de consola.
Las 29 pruebas compartidas aprobaron por hito; la revisión final alcanzó 33.
Diez pasos de consola dieron resultados idénticos entre las dos versiones.

2. Introducción y objetivos

El trabajo se centra en mejorar un módulo acotado sin ampliar sus funciones por acumulación. Para ello se creó un prototipo expresamente para esta actividad. No se lo presenta como un sistema institucional anterior ni como una solución de producción. Su organización inicial, monolítica y simple, permite analizar dificultades reales de mantenimiento sin introducir fallos funcionales a propósito.

El objetivo general es mostrar una relación verificable entre diagnóstico, cambio y prueba. Los objetivos concretos son separar responsabilidades, reducir duplicaciones, simplificar datos de entrada y explicitar las reglas de estados. Cada mejora se vincula con una debilidad y con registros de ejecución reales.

Este documento corresponde al trabajo grupal de Gevont Utmazian y Martín Maine, comisión M25 C4-08. El uso de inteligencia artificial como apoyo se declara en la sección 11 y la evidencia se adjunta en el anexo ZIP. Los datos del sistema son sintéticos: no se utilizan datos personales reales.

2.1 Integrantes, modalidad y participación

La modalidad es grupal. La tabla recoge el reparto de participación y las tareas de cada integrante informados por el equipo.

Integrante	Participación declarada	Detalle de aportes
Gevont Utmazian	56 %	Diseño del sistema y de la persistencia en SQLite. Diagnóstico de los problemas de diseño, aplicación de las refactorizaciones y comparación del comportamiento entre versiones.
Martín Maine	44 %	Descripción del caso de uso y del alcance. Preparación de ejemplos de consola, organización de las evidencias de prueba y revisión de la documentación de uso.

3. Sistema base y alcance

El sistema representa el registro de solicitudes por parte de un operador de una institución educativa. El estudiante es el solicitante; el responsable representa a una persona o área que atiende el trámite. En este prototipo ninguno inicia sesión: el operador usa una consola local. No se confundieron actores del dominio con cuentas autenticadas.

Estructura	Datos principales	Relación
Estudiante	id, nombre, email único	Uno a muchas solicitudes
Responsable	id, nombre	Cero o uno por solicitud
Solicitud	estudiante, tipo, prioridad, asunto, descripción, estado, responsable	Referencias protegidas por claves foráneas
Estado	pendiente, en_proceso, resuelta, cancelada	Valor del dominio; no tabla adicional

Se implementan ocho operaciones de consola: registrar estudiante, registrar responsable, registrar solicitud, asignar, cambiar estado, consultar, listar con filtros y reportar. Tipos: certificado, equivalencia e inscripcion. Prioridades: baja, media y alta. Todas las solicitudes nuevas comienzan pendientes.

No se incluyen adjuntos, autenticación, eliminación, notificaciones, plazos ni uso simultáneo multiusuario. La prioridad clasifica, pero no ordena automáticamente una cola. El reporte cuenta solicitudes por estado. La búsqueda opera sobre el asunto y trata el texto de forma literal.

3.1 Tecnología y ejecución

Python permite resolver consola, pruebas y persistencia con su biblioteca estándar. SQLite usa un archivo local y no requiere servidor [R2]. unittest organiza fixtures y casos automatizados [R1]. Se usó Python 3.10.0 disponible en el equipo; no se instaló pytest porque el intento de descarga fue bloqueado por la red. Esta adaptación fue técnica, no un requisito de la cátedra.

```
python -m unittest discover -s tests -v
python demo.py
python cli.py --db prueba.db reporte
```

La versión inicial se identifica con estado-inicial y se conserva también en versiones/inicial. El esquema de base no cambia, por lo que la comparación no necesita migraciones. Los datos usados en la demostración son ficticios.

4. Calidad externa e interna

4.1 Comparación de calidad externa e interna

Situación	Observación externa	Observación interna
Registrar una solicitud válida	Devuelve un identificador y conserva los datos	Una rama combina consultas previas, validaciones y SQL de inserción
Rechazar datos incompletos	Mensaje específico y ninguna solicitud persistida	La validación de texto se repite
Cambiar estado	Impide saltar de pendiente a resuelta	Las reglas están embebidas en una expresión booleana
Filtrar y reportar	Devuelve filas ordenadas y totales consistentes	Literales y acceso a datos viven en el mismo despachador

Las pruebas externas caracterizan el comportamiento, no certifican por sí solas mantenibilidad, rendimiento o seguridad integral. Los 29 casos iniciales se ejecutaron con `python -m unittest discover -s tests -v`; el registro auténtico está en `evidencias/pruebas/01_inicial.txt`.

4.2 Relación con la evolución futura

Desde el punto de vista del usuario, la versión inicial es correcta en los casos cubiertos: registra, asigna, cambia estado, filtra y reporta. La dificultad aparece al cambiar el sistema. Modificar el ciclo de estados exige editar una expresión booleana dentro de `operar`; agregar una operación obliga a ampliar el mismo despachador; y probar una regla requiere montar SQLite. Esos costos no se ven en la consola, pero determinan cuánto cuesta cada cambio y cuánto riesgo conlleva. La sección 5 detalla las ocho debilidades que explican esa diferencia y la sección 7 muestra cómo se redujeron.

5. Deuda técnica y code smells

5.1 Diagnóstico documentado

La referencia estable es `estado-inicial:sistema.py`. El prototipo fue construido para el TPI con una organización monolítica deliberadamente simple. No se presenta como software heredado de una institución. Funciona en los 29 casos de contrato ejecutados, pero modificarlo exige conocer SQL, reglas de negocio y el despachador de operaciones en una misma clase.

ID	Debilidad y ubicación	Evidencia literal	Impacto y riesgo	Prioridad	Corrección prevista
CS01	Long Method: `Sistema.operar`	`def operar(self, op, **d)` contiene todas las ramas	Una edición puede afectar operaciones no relacionadas; cuesta revisar el flujo completo	Alta	R1 Extract Method
CS02	Duplicación: ramas estudiante/responsable	`if not isinstance(d.get("nombre"), str) or not d["nombre"].strip()`	Dos validaciones iguales pueden divergir	Media	R1 función de texto obligatorio
CS03	Clase con responsabilidades mezcladas: `Sistema`	`sqlite3.connect(ruta)` y `raise ValueError("Transición inválida")`	Persistencia y política de estados cambian por razones diferentes	Alta	R2 Move Method, R4 Extract Class
CS04	Long Parameter List: `registrar_solicitud`	`estudiante_id, tipo, prioridad, asunto, descripcion, responsable_id=None`	Es fácil intercambiar cadenas posicionales del mismo tipo	Media	R3 Introduce Parameter Object
CS05	Literales de dominio repetidos: solicitud/listar	`("baja", "media", "alta")`	Un valor nuevo requiere cambios en puntos separados	Media	R4 constantes compartidas
CS06	Condicional compuesto: cambio de estado	`(r["estado"] == "pendiente" and ...) or (...)`	Al ampliar el ciclo es fácil habilitar una transición incorrecta	Alta	R4 tabla de transiciones
CS07	Nombres abreviados: listado y conexión	`q`, `p`, `r`, `d`, `self.c`	La lectura depende de reconstruir el significado local	Baja	R1 nombres de parámetros y R2 repositorio
CS08	Acoplamiento a la base de datos: creación y consultas	`self.c.execute(...)` dentro de reglas de negocio	Probar una política exige montar SQLite; cambiar persistencia afecta el servicio	Alta	R2 acceso encapsulado y R4 inyección de repositorio

CS03 describe la falta de cohesión de la clase; CS08, la dependencia concreta de sus reglas. Están relacionados, pero sus consecuencias y verificaciones son diferentes. Ninguno implica que el programa sea inseguro por definición: las consultas ya usan parámetros y las claves foráneas están activadas.

La prioridad alta se reserva para cambios que afectan reglas o varias operaciones. La media corresponde a duplicaciones y firmas que multiplican el costo del cambio. La baja corresponde a legibilidad localizada. No se asignaron tiempos ni costos económicos ficticios.

5.2 Ubicaciones comprobables

CS01 - versiones/inicial/sistema.py:31

```
def operar(self, op, **d):
    if op == "estudiante":
        if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
```

CS02 - versiones/inicial/sistema.py:33

```
            if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
                raise ValueError("Nombre obligatorio")
            if not isinstance(d.get("email"), str) or "@" not in d["email"] or not d["email"].strip():
```

CS03 - versiones/inicial/sistema.py:7

```
        self.c = sqlite3.connect(ruta)
        self.c.row_factory = sqlite3.Row
        self.c.execute("PRAGMA foreign_keys = ON")
```

CS04 - versiones/inicial/sistema.py:25

```
    def registrar_solicitud(self, estudiante_id, tipo, prioridad, asunto, descripcion,
                            responsable_id=None):
        return self.operar("solicitud", estudiante_id=estudiante_id, tipo=tipo,
```

CS05 - versiones/inicial/sistema.py:55

```
            if d.get("prioridad") not in ("baja", "media", "alta"):
                raise ValueError("Prioridad inválida")
            if not isinstance(d.get("asunto"), str) or not d["asunto"].strip():
```

CS06 - versiones/inicial/sistema.py:83

```
            if not ((r["estado"] == "pendiente" and d.get("estado") in ("en_proceso", "cancelada"))
                    or (r["estado"] == "en_proceso" and d.get("estado") in ("resuelta", "cancelada"))):
                raise ValueError("Transición inválida")
```

CS07 - versiones/inicial/sistema.py:95

```
            q = "SELECT * FROM solicitudes WHERE 1=1"
            p = []
            if d.get("estado") is not None:
```

CS08 - versiones/inicial/sistema.py:51

```
            if not self.c.execute("SELECT id FROM estudiantes WHERE id=?",
    (d.get("estudiante_id"),)).fetchone():
                raise ValueError("Estudiante inexistente")
            if d.get("tipo") not in ("certificado", "equivalencia", "inscripcion"):
```

6. SOLID, Clean Code y decisiones

SOLID aplicado con límites explícitos

Principio	Problema y cambio	Alcance real
SRP	Sistema mezclaba SQL y reglas. RepositorioSQLite administra almacenamiento; dominio concentra reglas; cli presenta resultados	Aplicado directamente. El servicio coordina casos de uso
OCP	El ciclo de estados estaba escrito como expresión booleana. TRANSICIONES permite cambiar relaciones sin editar el algoritmo de validación	Mejora localizada: una operación nueva todavía exige modificar el despachador. No se afirma cumplimiento absoluto
LSP	No hay jerarquía de herencia de producción que requiera sustitución de subclases	No se fuerza. La prueba de un repositorio mínimo demuestra sustitución solamente para reporte, no equivalencia total de repositorios
ISP	No se agregó una interfaz universal con operaciones ajenas al caso de uso	No se introducen interfaces nominales. El reporte utiliza únicamente contar_por_estado; una división mayor sería innecesaria para este alcance
DIP	El constructor acepta un repositorio; las reglas ya no ejecutan SQL	Aplicación parcial mediante inyección y contrato estructural. Sistema todavía importa SQLite como opción predeterminada; un composition root independiente sería una mejora futura

Clean Code se aplica mediante nombres descriptivos, validación reutilizable, métodos acotados y mensajes estables. Se mantiene una firma antigua con seis parámetros como adaptador: retirarla rompería compatibilidad. El código nuevo puede usar guardar_solicitud(NuevaSolicitud(...)) con nombres explícitos.

Decisiones técnicas

1. Python y SQLite permiten ejecutar el sistema sin servidor ni servicios externos. Las tres tablas y sus relaciones cubren el alcance sin un ORM.

2. unittest reemplazó la propuesta inicial pytest: pytest no estaba instalado y la instalación falló por restricciones de red. Se conservaron los mismos casos e intenciones. No se declara una ejecución de pytest.

3. Python 3.10.0 fue el intérprete disponible y efectivamente usado. La propuesta 3.11 no era un requisito obligatorio. No se recomienda instalar una versión antigua para producción.

4. Las transacciones cubren cada escritura. No se implementó un flujo transaccional multiusuario entre lectura y actualización: la aplicación es local, de un operador.

5. Se mantienen validación básica de email, búsqueda SQL y reglas iniciales. Corregirlas ahora sería un cambio funcional que debe separarse de la refactorización.

6. No se eliminó la API operar ni se alteró el esquema de SQLite. Las salidas JSON de diez pasos fueron idénticas entre ambas consolas.

7. Los datos de ejemplo son sintéticos. El nombre académico sólo aparece en documentación y no se incorpora como dato de estudiante del sistema.

8. No se calcularon cobertura ni métricas de complejidad ciclomática con herramientas externas. Las métricas estructurales del anexo se obtienen con ast y no son una puntuación de calidad.

Relaciones y componentes

Un estudiante puede tener varias solicitudes. Una solicitud tiene exactamente un estudiante y cero o un responsable. Un responsable puede atender varias solicitudes. La prioridad y el tipo clasifican; el estado representa el avance. La consola llama al servicio; éste utiliza reglas de dominio y el repositorio; únicamente el repositorio conoce SQL.

7. Refactoring seguro

Las cuatro transformaciones se hicieron en commits reales, en el orden del historial. Las pruebas preceden a los cambios y se repiten por hito. Los fragmentos siguientes se extraen del código; cuando son parciales se indica su rango. No se alteraron los registros históricos para aparentar trabajo previo.

R1 - Extract Method y validación compartida

CS01, CS02 y parte de CS07. Commit 32d2492. Se dividió el despachador en operaciones y se extrajo la validación repetida del nombre. SRP y legibilidad: cada caso de uso se inspecciona por separado. La interfaz operar y los errores permanecieron. Ver 02_extract_method.txt: 29 pruebas aprobadas.

versiones/inicial/sistema.py, líneas 31-45.

```
    def operar(self, op, **d):
        if op == "estudiante":
            if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
                raise ValueError("Nombre obligatorio")
            if not isinstance(d.get("email"), str) or "@" not in d["email"] or not d["email"].strip():
                raise ValueError("Email inválido")
            try:
                with self.c:
                    r = self.c.execute("INSERT INTO estudiantes(nombre,email) VALUES (?,?)",
                                       (d["nombre"].strip(), d["email"].strip().lower()))
                return r.lastrowid
            except sqlite3.IntegrityError as exc:
                raise ValueError("Email ya registrado") from exc
        elif op == "responsable":
            if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
```

sistema.py, líneas 19-28.

```
    def operar(self, operacion, **datos):
        acciones = {
            "estudiante": self._estudiante, "responsable": self._responsable,
            "solicitud": self._solicitud, "asignar": self._asignar,
            "estado": self._estado, "consultar": self._consultar,
            "listar": self._listar, "reporte": self._reporte,
        }
        if operacion not in acciones:
            raise ValueError("Operación desconocida")
        return acciones[operacion](datos)
```

dominio.py, líneas 25-27.

```
def validar_texto(valor, mensaje):
    if not isinstance(valor, str) or not valor.strip():
        raise ValueError(mensaje)
```

R2 - Move Method / reubicación de persistencia

CS03 y CS08. Commit 1cfd75c: conexión, esquema y transacciones se movieron al repositorio. El registro 03_move_method.txt conserva 29 aprobadas. La revisión detectó que el servicio todavía contenía SQL; a379c39 completó el encapsulamiento con métodos semánticos. Esa diferencia se declara para no atribuir al primer paso una separación que todavía no tenía. SRP y DIP parcial.

versiones/inicial/sistema.py, líneas 6-20.

```
    def __init__(self, ruta=":memory:"):
        self.c = sqlite3.connect(ruta)
        self.c.row_factory = sqlite3.Row
        self.c.execute("PRAGMA foreign_keys = ON")
        self.c.executescript("""
        CREATE TABLE IF NOT EXISTS estudiantes (
            id INTEGER PRIMARY KEY, nombre TEXT NOT NULL, email TEXT NOT NULL UNIQUE);
        CREATE TABLE IF NOT EXISTS responsables (
            id INTEGER PRIMARY KEY, nombre TEXT NOT NULL);
        CREATE TABLE IF NOT EXISTS solicitudes (
            id INTEGER PRIMARY KEY, estudiante_id INTEGER NOT NULL REFERENCES estudiantes(id),
            tipo TEXT NOT NULL, prioridad TEXT NOT NULL, asunto TEXT NOT NULL,
            descripcion TEXT NOT NULL, estado TEXT NOT NULL,
            responsable_id INTEGER REFERENCES responsables(id));
        """)
```

repositorio.py, líneas 42-47.

```
    def agregar_solicitud(self, nueva):
        return self._insertar("""INSERT INTO solicitudes
            (estudiante_id,tipo,prioridad,asunto,descripcion,estado,responsable_id)
            VALUES (?,?,?,?,?,'pendiente',?)""",
            (nueva.estudiante_id, nueva.tipo, nueva.prioridad,
             nueva.asunto.strip(), nueva.descripcion.strip(), nueva.responsable_id))
```

sistema.py, líneas 7-8.

```
    def __init__(self, ruta=":memory:", repositorio=None):
        self.repositorio = repositorio if repositorio is not None else RepositorioSQLite(ruta)
```

R3 - Introduce Parameter Object

CS04. Commit b5eb55f. NuevaSolicitud reúne los seis datos de una solicitud; guardar_solicitud recibe ese objeto. La API posicional original se conserva como adaptador, por lo que la simplificación no exige romper clientes. No se presenta como eliminación total de la firma larga. Registro 04_parameter_object.txt: 29 aprobadas. La prueba adicional test_objeto_parametros comprueba la nueva vía.

versiones/inicial/sistema.py, líneas 25-29.

```
    def registrar_solicitud(self, estudiante_id, tipo, prioridad, asunto, descripcion,
                            responsable_id=None):
        return self.operar("solicitud", estudiante_id=estudiante_id, tipo=tipo,
                           prioridad=prioridad, asunto=asunto, descripcion=descripcion,
                           responsable_id=responsable_id)
```

dominio.py, líneas 16-22.

```
class NuevaSolicitud:
    estudiante_id: int
    tipo: str
    prioridad: str
    asunto: str
    descripcion: str
    responsable_id: int | None = None
```

sistema.py, líneas 46-58.

```
    def guardar_solicitud(self, nueva: NuevaSolicitud):
        if not self.repositorio.existe_estudiante(nueva.estudiante_id):
            raise ValueError("Estudiante inexistente")
        if nueva.tipo not in TIPOS:
            raise ValueError("Tipo inválido")
        if nueva.prioridad not in PRIORIDADES:
            raise ValueError("Prioridad inválida")
        validar_texto(nueva.asunto, "Asunto obligatorio")
        validar_texto(nueva.descripcion, "Descripción obligatoria")
        if nueva.responsable_id is not None:
            if not self.repositorio.existe_responsable(nueva.responsable_id):
                raise ValueError("Responsable inexistente")
        return self.repositorio.agregar_solicitud(nueva)
```

R4 - Decompose Conditional y reglas separadas

CS05 y CS06; completa CS03, CS07 y CS08. Commit a379c39. La condición compuesta se reemplazó por una consulta a TRANSICIONES, con catálogos compartidos. La creación del repositorio también materializa Extract Class. Se preservan exactamente las cuatro transiciones permitidas. OCP localizado: cambia la tabla, no el algoritmo. Registro 05_separar_reglas.txt: 29 aprobadas; la revisión posterior recorre los 16 pares.

versiones/inicial/sistema.py, líneas 79-89.

```
        elif op == "estado":
            r = self.c.execute("SELECT * FROM solicitudes WHERE id=?", (d.get("id"),)).fetchone()
            if r is None:
                raise ValueError("Solicitud inexistente")
            if not ((r["estado"] == "pendiente" and d.get("estado") in ("en_proceso", "cancelada"))
                    or (r["estado"] == "en_proceso" and d.get("estado") in ("resuelta", "cancelada"))):
                raise ValueError("Transición inválida")
            with self.c:
                self.c.execute("UPDATE solicitudes SET estado=? WHERE id=?", (d["estado"], d["id"]))
            return None
        elif op == "consultar":
```

dominio.py, tabla de transiciones.

```
TRANSICIONES = {
    "pendiente": ("en_proceso", "cancelada"),
    "en_proceso": ("resuelta", "cancelada"),
    "resuelta": (),
    "cancelada": (),
}
ESTADOS = tuple(TRANSICIONES)
```

dominio.py, líneas 30-32.

```
def validar_transicion(actual, destino):
    if destino not in TRANSICIONES.get(actual, ()):
        raise ValueError("Transición inválida")
```

7.5 Indicadores estructurales

Versión	Líneas de operar	Funciones en sistema.py
Inicial	88	4
Final	10	13

Se contaron líneas físicas del nodo AST y funciones declaradas. El número total de funciones puede aumentar al extraer responsabilidades. Estos datos no prueban una mejora de rendimiento ni constituyen una métrica de calidad por sí solos.

8. Pruebas y verificación

Los identificadores P01-P29 organizan los casos por funcionalidad; unittest los imprime ordenados por nombre de método. Antes: registro 01_inicial.txt. Después: 05_separar_reglas.txt y 07_revision_optimizada.txt. En las columnas se indica coincidencia con el resultado esperado, no salidas fabricadas.

ID	Caso y entrada	Resultado esperado	Antes	Después
P01	Estudiante válido: Demo; demo@example.test	ID 1	Coincide	Coincide
P02	Email duplicado: Mismo email con mayúsculas	Email ya registrado	Coincide	Coincide
P03	Nombre vacío: nombre vacío	Nombre obligatorio	Coincide	Coincide
P04	Email inválido: incorrecto	Email inválido	Coincide	Coincide
P05	Solicitud válida: Certificado, alta, estudiante 1	Fila completa; pendiente	Coincide	Coincide
P06	Asunto vacío: espacios	Asunto obligatorio; no persiste	Coincide	Coincide
P07	Descripción vacía: cadena vacía	Descripción obligatoria; no persiste	Coincide	Coincide
P08	Tipo inválido: otro	Tipo inválido; no persiste	Coincide	Coincide
P09	Prioridad inválida: urgente	Prioridad inválida; no persiste	Coincide	Coincide
P10	Estudiante inexistente: ID 999	Estudiante inexistente	Coincide	Coincide
P11	Responsable inicial inexistente: ID 999	Responsable inexistente	Coincide	Coincide
P12	Asignar responsable: Solicitud 1; responsable 1	responsable_id=1	Coincide	Coincide
P13	Responsable al crear: Responsable válido	Asignación persistida	Coincide	Coincide
P14	Ciclo normal: pendiente; en_proceso; resuelta	Dos cambios válidos	Coincide	Coincide
P15	Salto inválido: pendiente a resuelta	Transición inválida; conserva estado	Coincide	Coincide
P16	Estado repetido: pendiente a pendiente	Transición inválida	Coincide	Coincide
P17	Estado desconocido: desconocido	Transición inválida	Coincide	Coincide
P18	Cancelación terminal: cancelada a en_proceso	Transición inválida	Coincide	Coincide
P19	Filtros y búsqueda: alta, pendiente, CONSTANCIA y %	Resultados combinados y búsqueda literal	Coincide	Coincide
P20	Filtro estado inválido: otro	Estado inválido	Coincide	Coincide
P21	Filtro prioridad inválida: otra	Prioridad inválida	Coincide	Coincide
P22	Reporte: Base vacía y una pendiente	Total 0 y 1; estados en cero incluidos	Coincide	Coincide
P23	Consulta inexistente: ID 999	Solicitud inexistente	Coincide	Coincide
P24	Cambio inexistente: ID 999	Solicitud inexistente	Coincide	Coincide
P25	Asignación inexistente: ID 999	Solicitud inexistente	Coincide	Coincide
P26	Responsable al asignar inexistente: ID 999	Rechazo; mantiene sin responsable	Coincide	Coincide
P27	Persistencia: Cerrar y reabrir archivo	Mismo asunto y solicitud	Coincide	Coincide

ID	Caso y entrada	Resultado esperado	Antes	Después
P28	SQL como dato: Texto con DROP TABLE	Se guarda literal; tabla conservada	Coincide	Coincide
P29	Operación desconocida: borrar_todo	Operación desconocida	Coincide	Coincide

Verificaciones añadidas en revisión

R01 recorre 16 pares de estados, con cuatro permitidos y doce rechazados. R02 prueba NuevaSolicitud. R03 inyecta un doble de conteos para reporte sin SQLite. R04 ejecuta una CLI con ID inexistente y verifica JSON, código 1 y ausencia de stderr. Estas cuatro pruebas no existían en la suite inicial; no se presentan como comparación antes/después.

La versión final ejecutó 33 pruebas. La matriz de 16 pares cuenta como un método con subcasos. demo.py realiza además diez pasos por versión y compara salida y código. No se midió cobertura; no se promete verificación exhaustiva.

Método

Cada prueba de servicio abre SQLite en memoria y cierra la conexión. La persistencia se prueba con un archivo temporal reabierto. Las pruebas de consola usan procesos reales y bases temporales. No se usa una base de producción ni se limpian archivos del usuario. Los registros son salidas capturadas de los comandos ejecutados; no son capturas recreadas gráficamente.

8.1 Registros de ejecución

01_inicial.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.128s

OK
```

02_extract_method.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.102s

OK
```

03_move_method.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.106s

OK
```

04_parameter_object.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.112s

OK
```

05_separar_reglas.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.104s

OK
```

06_revision_final.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 33 tests in 0.393s

OK
```

07_revision_optimizada.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 33 tests in 0.382s

OK
```

8.2 Demostración antes/después

La ejecución de demo.py informó IGUALES para diez pasos. Ejemplos de resultados realmente capturados:


```
{
  "argumentos": [
    "solicitud",
    "--estudiante-id",
    "1",
    "--tipo",
    "certificado",
    "--prioridad",
    "alta",
    "--asunto",
    "Constancia regular",
    "--descripcion",
    "Beca ficticia"
  ],
  "salida": {
    "ok": true,
    "resultado": 1
  },
  "codigo": 0
}

{
  "argumentos": [
    "consultar",
    "--id",
    "1"
  ],
  "salida": {
    "ok": true,
    "resultado": {
      "id": 1,
      "estudiante_id": 1,
      "tipo": "certificado",
      "prioridad": "alta",
      "asunto": "Constancia regular",
      "descripcion": "Beca ficticia",
      "estado": "resuelta",
      "responsable_id": 1
    }
  },
  "codigo": 0
}
```

```
{
  "argumentos": [
    "estado",
    "--id",
    "1",
    "--estado",
    "pendiente"
  ],
  "salida": {
    "ok": false,
    "error": "Transici├│n inv├ílida"
  },
  "codigo": 1
}

{
  "argumentos": [
    "reporte"
  ],
  "salida": {
    "ok": true,
    "resultado": {
      "total": 1,
      "por_estado": {
        "pendiente": 0,
        "en_proceso": 0,
        "resuelta": 1,
        "cancelada": 0
      }
    }
  },
  "codigo": 0
}
```

9. Code review y criterios de aceptación

Checklist técnico aplicado

Revisión técnica del código final con un checklist de 14 criterios, realizada con asistencia de IA el 01/11/2026. Cada criterio se valora contra el código y las pruebas del repositorio; el mismo checklist está en docs/checklist_code_review.md.

Criterio	Resultado	Evidencia o límite
Legibilidad y nombres	Conforme con límite	Métodos descriptivos; API compatible conserva diccionarios
Responsabilidades y cohesión	Conforme	Consola, servicio, dominio y repositorio separados
Duplicación	Conforme con límite	Validación de texto común; catálogo centralizado. Literal pendiente sigue en SQL de inserción
Complejidad	Conforme al alcance	Operaciones separadas; estados definidos por tabla
Validaciones	Conforme al contrato	Rechazos no crean solicitudes; mensajes conservados
Manejo de errores	Parcial	Errores de dominio controlados; fallos de disco/SQLite no traducidos en la CLI
Acoplamiento y SOLID	Parcial justificado	Inyección disponible; dependencia predeterminada SQLite permanece
Persistencia	Conforme	Reapertura probada, claves foráneas y transacciones
Seguridad básica	Conforme al alcance	SQL parametrizado, datos sintéticos; sin autenticación
Pruebas	Conforme	29 casos de contrato y 4 revisiones; matriz de 16 pares de estados
Organización	Conforme	Versión inicial conservada; evidencias separadas
Documentación	Conforme	README, diagnóstico, refactorings, registro de IA e informe
Mantenibilidad	Mejorada con deuda residual	No se atribuye escalabilidad ni preparación para producción

Observaciones y acciones

ID	Archivo	Severidad	Observación y recomendación	Acción y estado
CR01	sistema.py, hito Move Method	Media	Mover sólo conexión dejaba SQL en las reglas. Encapsular también consultas	Corregido en a379c39: métodos semánticos del repositorio
CR02	tests/test_contrato.py	Media	Suite de contrato no recorre todos los pares de estados	Se añadió test_matriz_completa_transiciones: 16 subcasos, sin cambiar producción
CR03	cli.py	Media	Las pruebas del servicio no garantizan salida de consola	demo.py compara diez pasos reales; prueba adicional verifica error y código 1
CR04	sistema.py	Baja	Inyección sin prueba de sustitución	Se agregó doble mínimo para reporte; no se lo presenta como repositorio completo
CR05	dominio.py, repositorio.py	Baja	Estado inicial pendiente también aparece como literal SQL	Deuda residual aceptada; no se inventa eliminación total de literales
CR06	cli.py	Media	Error de disco/base dañada puede generar traceback	Aceptado como límite: mejora funcional independiente, documentada en README
CR07	tests/test_contrato.py	Media	Asserts nativos pueden omitirse con python -O	Reemplazados por aserciones unittest y ejecutada la suite con optimización

Criterios de aceptación técnica

La suite compartida debe aprobar en el estado inicial y en cada refactoring. La demostración debe devolver igualdad exacta. No debe existir SQL en el servicio final. El proyecto debe poder descargarse y ejecutar sus pruebas sin instalar paquetes. Deben existir los seis tags de hitos. El repositorio debe permitir recuperar el historial completo.

10. Conclusiones y mejoras futuras

El resultado más relevante es la separación entre reglas de aplicación y acceso a datos. Antes, el mismo método contenía validaciones, consultas y cambios de estado. Después, el servicio expresa las operaciones y el repositorio administra las consultas. La tabla de transiciones permite inspeccionar el ciclo sin reconstruir una expresión booleana extensa.

El proceso también mostró que una extracción superficial puede no resolver el problema completo. Mover únicamente la conexión dejó SQL dentro del servicio; fue necesario encapsular las consultas para completar esa mejora. La revisión de pruebas detectó otra debilidad: las aserciones nativas podían omitirse con optimización. Se sustituyeron y se verificó la suite con python -O.

No se afirma que el sistema esté libre de errores. La evidencia respalda el contrato cubierto. Quedan pendientes mensajes para errores técnicos, validación de email más estricta, búsqueda Unicode completa, autenticación y concurrencia. Esos cambios deberían planificarse como mejoras funcionales separadas y con pruebas propias.

Aprendizajes

El trabajo mostró que un sistema puede cumplir su contrato externo y aun así concentrar demasiados motivos de cambio. Separar servicio, dominio y repositorio, y explicitar las transiciones de estado, hizo más localizable cada modificación. Las pruebas compartidas entre ambas versiones permitieron controlar que el comportamiento observable se mantuviera, y el diagnóstico con ubicación, impacto y prioridad permitió decidir qué corregir primero.

Dificultades

Mover sólo la conexión dejó SQL dentro del servicio y obligó a un segundo paso (R2 completado en a379c39). Además, pytest no estaba disponible en el entorno y la suite se migró a unittest.

Límites

La evidencia respalda el contrato cubierto por 33 pruebas y diez pasos de consola; no se midió cobertura. La revisión técnica fue asistida por IA, no independiente. El sistema es un prototipo sin autenticación ni concurrencia.

Mejoras futuras

Traducir errores técnicos de disco o SQLite en la consola, validar emails con mayor rigor, soportar búsqueda Unicode completa, incorporar autenticación y concurrencia, y sumar una revisión cruzada de código con otro equipo.

11. Uso de inteligencia artificial

Herramienta: ChatGPT/Codex en Work Mode. Fecha conocida: 01/11/2026. Intervención sustancial: lectura de requisitos, diseño del prototipo, programación, refactorización, generación y ejecución de pruebas, revisión técnica, documentación y preparación de defensa. No se limita a corrección ortográfica.

Intercambios disponibles

1. Prompt maestro suministrado por el usuario: copia íntegra del texto extraído en `prompt_maestro.txt` (se normalizaron saltos de línea). Fuente: archivo Texto pegado(7).txt. Contiene alcance, criterios de redacción, etapas, evidencias y prohibición de publicar sin autorización.

2. Instrucción del usuario en esta sesión: «Ahi te cargue el tpi, te di el prompt maestro para que empieces a trabajar en ello.» Se utilizó como autorización para construir y verificar los entregables locales.

3. Respuesta de ejecución del asistente: lectura completa de consigna y complementos; selección del caso sugerido. Se implementaron versiones comparables, con commits que identifican al asistente.

4. Incidencia real: pytest no disponible; pip no pudo conectarse por restricciones de red. Se migró la suite a unittest y se ejecutó. No se conserva una captura de ese error; este registro es una descripción de la incidencia, no una transcripción literal de la consola.

5. Validación: registros completos de las ejecuciones por hito en evidencias/pruebas; comparación de consolas en evidencias/demo_comparada.json; revisión final en checklist_code_review.md.

Los puntos 3 a 5 resumen el proceso real; no son una transcripción completa de la sesión. Los diffs y commits permiten inspeccionar los cambios realizados. El prompt maestro completo y este registro se adjuntan en el anexo ZIP (docs/prompt_maestro.txt y docs/registro_ia.md).

Responsabilidad y límites

La IA se usó como apoyo, con un grado de intervención amplio que se declara aquí. Gevont Utmazian y Martín Maine asumen la autoría del trabajo entregado y lo explican con voz propia en el video grupal. Las conclusiones y la defensa corresponden al equipo; no se utilizó voz generada por IA.

12. Referencias y recursos

[R1] Python Software Foundation. Documentación de unittest, Python 3.10. Consulta: 01/11/2026. https://docs.python.org/3.10/library/unittest.html

[R2] Python Software Foundation. Documentación de sqlite3, Python 3.10. Consulta: 01/11/2026. https://docs.python.org/3.10/library/sqlite3.html

[R3] Martin Fowler. Refactoring Catalog: Extract Function (alias Extract Method). Consulta: 01/11/2026. https://refactoring.com/catalog/extractFunction.html

[R4] Martin Fowler. Refactoring Catalog: Introduce Parameter Object. Consulta: 01/11/2026. https://refactoring.com/catalog/introduceParameterObject.html

Los términos Extract Method e Introduce Parameter Object se emplean según esas entradas del catálogo [R3, R4]. Los fragmentos del informe son del proyecto, no copias de los ejemplos del sitio.

[C1] Cátedra de Metodología de Sistemas II. TP MS2V2.pdf, seis páginas, suministrado por el usuario. Fuente principal de requisitos.

[C2] Material complementario suministrado como Markdown pegado(1).md. Incluye orientaciones de entrega y defensa.

Repositorio GitHub: proyectofinal_met2.

Video grupal: ENLACE_AL_VIDEO (reemplazar por la URL real antes de regenerar el PDF).

Fecha de entrega: 08/11/2026. Participación de cada integrante: ver sección 2.1. Anexo ZIP: evidencias complementarias, registros de pruebas y registro de uso de IA.

Anexo A. Historial real de desarrollo

Extracto del historial de commits del desarrollo y la verificación. El historial completo y los tags (estado-inicial, etapa-diagnostico, etapa-refactoring, etapa-pruebas, etapa-docs, estadorefactorizado) están en el repositorio.

```
28ac710 | Crear prototipo académico y contrato de comportamiento
d62f552 | Verificar el prototipo con unittest disponible sin instalación
efcdcb4 | Documentar ocho debilidades y conservar la referencia inicial
32d2492 | Extraer operaciones y validación repetida de nombres
1cfd75c | Mover conexión y transacciones a repositorio SQLite inyectable
b5eb55f | Introducir NuevaSolicitud conservando la interfaz compatible
a379c39 | Encapsular consultas y descomponer reglas de dominio en tabla de estados
b3828db | Verificar consola equivalente, dominio y repositorio sustituible
85689e4 | Corregir aserciones para que no desaparezcan con optimización
```